

Agentic Workflow Plan — SDLC Integration

Worship Ministry App · Personal Learning Platform → Team Playbook

Purpose: Deep analysis of the current agentic state, what is missing, the quality control architecture, and a concrete roadmap to a fully autonomous multi-agent development workflow — one that could build a feature end-to-end with minimal human intervention, while maintaining production-grade quality controls.

Table of Contents

- 1. [What We Have — Honest Inventory](#)
- 2. [What Is Missing — The Gaps](#)
- 3. [Quality Control Architecture — Where Bugs Are Caught](#)
- 4. [The Target Workflow — What Full Agentic Looks Like](#)
- 5. [SDLC Integration — Every Phase Mapped](#)
- 6. [Day-to-Day Developer Experience](#)
- 7. [Role Plays](#)
 - [Building a Feature](#)
 - [Fixing a Bug](#)
 - [Testing a Feature](#)
- 8. [Roadmap — How to Get There](#)
- 9. [Team Introduction Playbook](#)

1. What We Have — Honest Inventory

1.1 Agent Infrastructure

Asset	Location	What It Does	Maturity
Skill library	.claude/skills/	9 reasoning personas (SDET, Staff Eng, PM, etc.)	Mature
Named agents	.claude/agents/	5 specialist agents (debugger, security-auditor, modal-guard, migration-reviewer, async-handler-auditor)	Partially used
Planning pipeline	.claude/prompts/	6-phase feature pipeline (definition → release)	Exists, rarely triggered
Persistent memory	MEMORY.md + topics	Cross-session learning, rule capture	Active
Project context	.claude/context/PROJECT-CONTEXT.md	Architecture, known gaps, multi-tenant state	Active
CLAUDE.md (root)	/CLAUDE.md	Non-negotiable rules, anti-patterns, code conventions	Active

1.2 Quality Infrastructure

Asset	Location	Scope	Gate Type
Unit tests	__tests__/unit/	Pure logic (dates, sorting, constants)	CI
Integration tests	__tests__/integration/	API routes, tenant isolation, auth	CI
Component tests	__tests__/components/	Modal forms, role-based rendering	CI + pre-commit

Intent tests	__tests__/intent/	22 tests, 8 critical user journeys	CI + pre-commit
Impact mapper	shadow-agent/mapper.ts	Maps source file → test files	Used by hooks
Pre-commit hook	.husky/pre-commit	Blocks commit if intent tests fail	Pre-commit
ESLint	.eslintrc	TypeScript strict, Next.js rules	CI + editor
Total: 705+ tests	—	—	—

1.3 Orchestration Patterns (Used Ad-Hoc, Not Codified)

This project has already used multi-agent orchestration — but informally, in conversation:

- **Debate pattern:** 3 personas debate architecture, converge on a decision (RTL vs Playwright)
- **Subagent delegation:** Plan in main session → implement in Bash subagent → verify in main session
- **Sequential specialization:** PM → Staff Eng → SDET → implement → audit

These patterns exist in session transcripts but are not documented as repeatable playbooks. That is the main gap between "we've done agentic" and "we have an agentic workflow."

1.4 What Is NOT Agentic Yet

- Claude edits files and has **no automatic feedback loop** — it doesn't know if it broke a user journey
- Specialist agents are **manually invoked** — no automatic triggering on events
- No **orchestrator agent** that plans and delegates a full feature without human prompting each step
- No **CI agent** that autonomously investigates failures and proposes fixes
- No **PR review agent** that validates code quality before merge

2. What Is Missing — The Gaps

Gap 1: No In-Session Feedback Loop (Highest Priority)

The problem: Claude edits `people/page.tsx`, moves to the next file. It has no way to know it just broke the Coordinator read-only journey until the developer notices or a test runs later.

What Anthropic does internally: Every tool call that modifies state is followed by a verification step. The agent doesn't declare completion until it has confirmed the edit achieves the intended outcome.

The fix: PostToolUse hook — intent guard fires after every Edit/Write on `src/`, runs impacted tests, blocks Claude on failure.

Status: Designed, not implemented.

Gap 2: Agents Are Manually Invoked

The problem: `tenant-security-auditor`, `modal-regression-guard`, `migration-safety-reviewer`, `async-handler-auditor` all exist as agent descriptions but are only used when the developer remembers to invoke them.

What gets missed: A new API route added without running `tenant-security-auditor`. A new modal added without running `modal-regression-guard`. A SQL migration applied without `migration-safety-reviewer`.

The fix: Wire agents to events via hooks:

- PostToolUse on `src/app/api/**` → auto-trigger `tenant-security-auditor`
- PostToolUse on any `page.tsx` → auto-trigger `modal-regression-guard`
- PostToolUse on `supabase/migrations/*.sql` → auto-trigger `migration-safety-reviewer`

Status: Not implemented.

Gap 3: No Documented Orchestration Pattern

The problem: We've done supervisor → worker delegation (debate agent → implementation agent → verification) but it's not a written, repeatable playbook. A new team member (or future session) can't replicate it.

The fix: Document the orchestration patterns in `.claude/context/` as named patterns with examples. Define: what the orchestrator's job is, how it delegates, what it verifies, when it escalates to human.

Status: Not documented.

Gap 4: Incomplete Intent Test Coverage

The problem: 22 intent tests cover 8 journeys across 5 pages. There are 11 admin pages and several untested journeys:

- Setlist page (create setlist, add songs, publish)
- Availability page (member submits, coordinator views)
- Handbook page (editor access, content update)
- Dashboard (data loads correctly per role)
- Audit log (events are recorded and visible)

The fix: Extend `__tests__/intent/` to cover all critical pages. Add to `shadow-agent/mapper.ts`.

Status: Partial.

Gap 5: No E2E Smoke Layer Against Staging

The problem: All 705+ tests run against mocked data. No test touches a real Supabase instance. If a cookie format changes, an RLS policy breaks, or a migration has a typo — nothing catches it until production.

The fix: 3-5 Playwright smoke tests against staging Supabase:

- Login → redirect to roster
- Admin creates a member (real DB write)
- Coordinator blocked from Settings

Status: Decided against in debate, but should be revisited once staging environment is stable.

Gap 6: No Automated PR Review Agent

The problem: PRs are manually reviewed. Claude's code improvement advisor is manually invoked.

The fix: A GitHub Actions workflow that calls Claude (via Anthropic API) on every PR to:

1. Run `code-improvement-advisor` on all changed files
2. Run `tenant-security-auditor` on any `api/` changes
3. Post a structured review comment to the PR
4. Fail the check if critical issues found

Status: Not implemented.

3. Quality Control Architecture — Where Bugs Are Caught

This is the most important section. Every quality control must be assigned to a specific gate. If it has no gate, it will slip to production.

The 6-Layer Quality Stack



• Error monitoring (Sentry / Supabase logs) • User feedback • Last line of defence – bugs here are expensive	
LAYER 5: POST-DEPLOY / STAGING • Playwright smoke tests (3–5 critical paths) [MISSING] • Deploy verification agent [MISSING] • Manual smoke by developer (current state)	
LAYER 4: CI / PR MERGE GATE • Full 705+ test suite (unit + integration + component) • Intent tests (22 user journeys) • ESLint + TypeScript build • Automated PR review agent [MISSING] • Tenant security audit on api/ changes [MISSING – automated]	
LAYER 3: PRE-COMMIT • Husky: npm run test:intent (22 intent tests) [EXISTS] • ESLint [EXISTS via CI, not pre-commit hook yet] • Migration safety check on *.sql files [MISSING]	
LAYER 2: IN-SESSION / PRE-CONTINUATION • PostToolUse intent guard hook [MISSING] • PostToolUse API security check [MISSING] • PostToolUse modal regression guard [MISSING] • PostToolUse migration safety review [MISSING]	← BIGGEST GAP
LAYER 1: RULES & GUARDRAILS • CLAUDE.md non-negotiable rules [EXISTS] • Role default = null pattern [DOCUMENTED] • try/catch/finally handler pattern [DOCUMENTED] • No placeholder comments in forms [DOCUMENTED] • Tenant-scoped query rules [DOCUMENTED]	

Quality Control Gap Map

Bug Type	Currently Caught At	Should Be Caught At	Gap
Blank modal form	Pre-commit (intent test)	Layer 2 hook	Hook fires before commit
Wrong canEdit logic	Pre-commit (intent test)	Layer 2 hook	Hook fires immediately
IDOR / missing tenant_id	Manual review (if remembered)	Layer 2 hook + Layer 4 CI	Auto-trigger on api/ edit
Async handler no try/catch	Manual review	Layer 2 hook (async-handler-auditor)	Auto-trigger on handler edit
Unsafe migration	Manual review (if remembered)	Layer 3 pre-commit	Auto-trigger on .sql file
Role default "Admin"	Pre-commit (if intent tests catch it)	Layer 2 hook	Hook fires immediately
Breaking DB schema	Layer 4 CI (integration tests)	Layer 2 + 3	Migration reviewer hook

Broken auth cookie	Nothing (mocked away)	Layer 5 staging smoke	Playwright smoke needed
Cross-tenant data leak	Layer 4 (tenant-isolation.test.ts)	Layer 2 hook	Auto-trigger on api/ edit

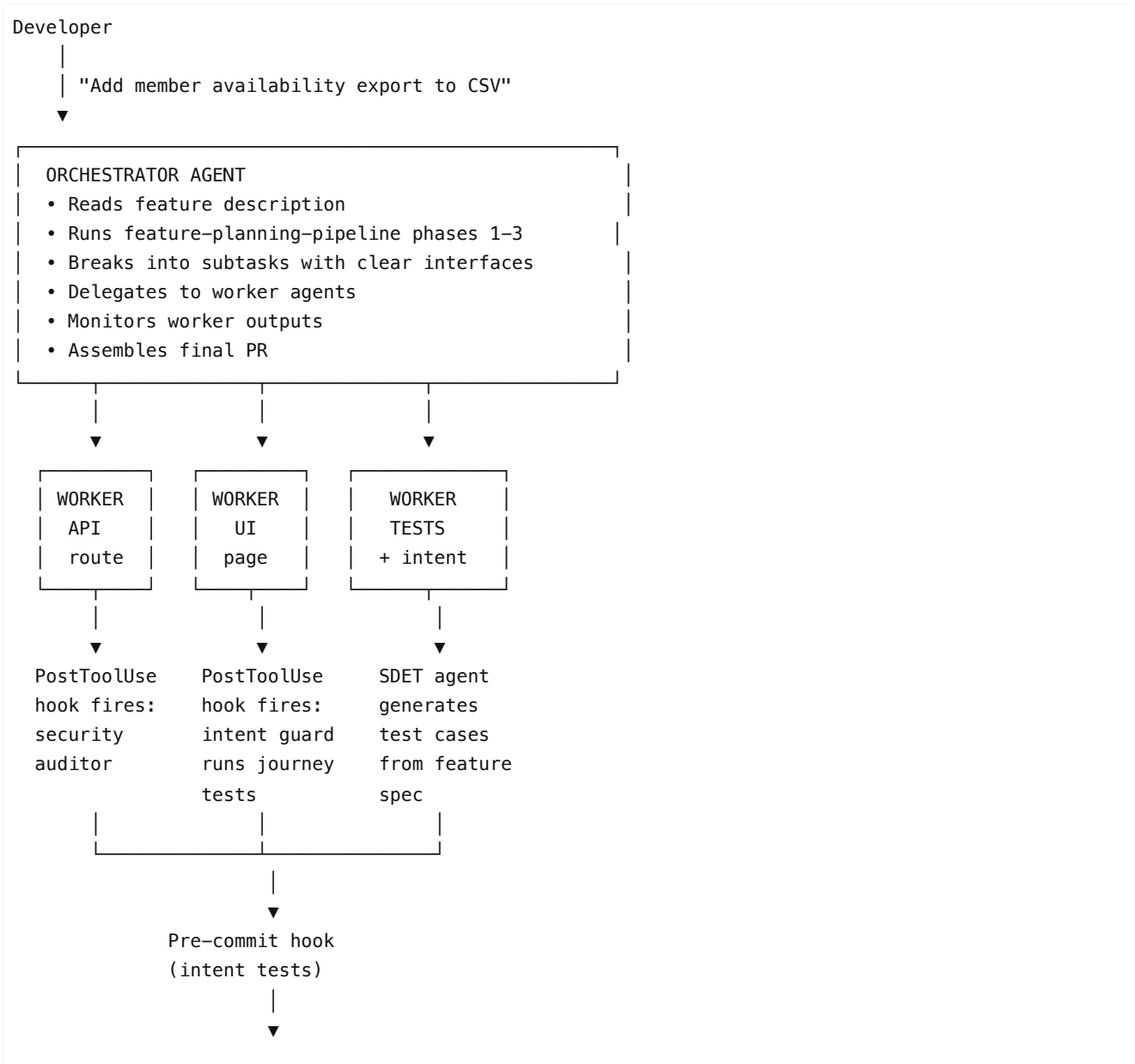
The Quality Gap in One Sentence

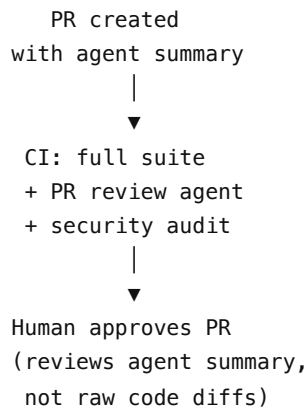
Layer 2 (in-session) is almost entirely empty. Claude can write 50 lines of broken code, commit them, and the first quality gate it hits is the pre-commit hook 10 minutes later. The PostToolUse hooks are the single most impactful addition to the quality stack.

4. The Target Workflow — What Full Agentic Looks Like

The goal: a developer describes a feature in one sentence. An orchestrator agent plans it, delegates to specialist workers, each worker self-validates their output, quality gates fire automatically at each transition, and the developer reviews a completed PR — not individual code changes.

The Target Architecture





What "Close to What Anthropic Does" Means

Anthropic's internal agents follow these patterns (from public talks and papers):

1. **Agents verify their own outputs** — after every action, the agent checks the result matches the intent
2. **Agents fail loudly and early** — a failing test is surfaced immediately, not discovered later
3. **Orchestrators plan, workers execute** — the planning agent never touches files; worker agents never make scope decisions
4. **Human escalation for irreversible actions** — pushing to main, running migrations, deleting data all require human sign-off
5. **Agents have bounded scope** — each agent has a specific job and refuses to do adjacent work

The worship app is approximately 40% of the way to this target. The hooks layer (Layer 2) is the biggest gap.

5. SDLC Integration — Every Phase Mapped

Phase 1: Discovery & Definition

Current state: Developer prompts Claude manually. PM thinking happens inline.

Target state:

Developer: "We need members to export their availability to CSV"

Orchestrator:

1. Invokes product-manager skill → writes user stories + acceptance criteria
2. Invokes systems-thinking skill → identifies dependencies (auth, RLS, tenant isolation)
3. Invokes ai-system-designer if AI features involved
4. Outputs: feature-spec.md with scope, non-goals, open questions
5. Pauses for human review of spec before implementation

Quality gate: Human approves spec before any code is written.

Phase 2: Architecture & Planning

Current state: Ad-hoc, or manually triggered via feature-planning-pipeline.md.

Target state:

Orchestrator (continuing from spec):

1. Invokes saas-architect skill → data model, API design, tenant isolation strategy
2. Invokes staff-engineer skill → implementation plan, file list, sequencing
3. Invokes SDET skill → test strategy, edge cases, risk areas
4. Outputs: implementation-plan.md with: files to create/edit, test files needed,

migration required (y/n), security surface (y/n)
5. Pauses for human approval of plan

Quality gate: Human approves architecture before implementation.

Phase 3: Implementation

Current state: Claude writes code file by file. No automatic validation.

Target state:

Orchestrator spawns parallel workers based on implementation plan:

Worker A (API):

- Creates/edits API route files
- PostToolUse hook → tenant-security-auditor fires on every api/ file
- PostToolUse hook → async-handler-auditor fires on every handler
- Reports: files created, hooks passed, ready for test

Worker B (UI):

- Creates/edits page component files
- PostToolUse hook → intent guard fires, runs impacted journey tests
- PostToolUse hook → modal-regression-guard fires on page.tsx files
- Reports: UI complete, all intent tests passing

Worker C (DB):

- Creates migration file
- PostToolUse hook → migration-safety-reviewer fires immediately
- Reports: migration reviewed, safe to apply

Orchestrator:

- Waits for all workers to report green
- Does NOT proceed to commit if any worker reports hook failure
- If failure: re-delegates to the responsible worker with the error context

Quality gates:

- PostToolUse hooks on every file edit (Layer 2)
 - Worker agents cannot mark themselves "done" with failing hooks
-

Phase 4: Testing

Current state: SDET agent manually invoked to define test strategy. Claude writes tests.

Target state:

Test Worker (spawned by orchestrator):

1. Reads feature spec + implementation plan
2. Invokes SDET skill → generates full test matrix
3. Writes unit tests for new business logic
4. Writes intent tests for new user journeys
5. Extends shadow-agent/mapper.ts for new files
6. Runs full intent suite: npm run test:intent
7. Runs targeted integration tests for new API routes
8. Reports: X unit tests, Y intent tests, all passing

Quality gate: Test worker cannot report "done" until all new tests are green AND coverage thresholds maintained.

Phase 5: Pre-Merge Review

Current state: Manual PR, manual review.

Target state:

Orchestrator:

1. Gathers all worker reports
2. Invokes code-improvement-advisor on all changed files
3. Invokes tenant-security-auditor on all api/ changes (full pass)
4. Generates PR body: feature summary, test coverage, hook results, agent reports
5. Creates PR via `gh pr create`
6. Posts PR review comment with structured agent summary

CI (GitHub Actions):

- Full test suite: 705+ tests
- `npm run test:intent`
- `npm run lint`
- `npm run build`
- Automated PR review agent (Claude via API) → posts structured comment

Human:

- Reviews agent summary in PR (not raw diffs)
- Approves or requests changes
- Merge

Quality gates:

- All CI checks must pass
- Automated PR review must not flag critical issues
- Human approval required before merge

Phase 6: Post-Merge

Current state: Nothing automated after merge.

Target state:

Post-merge agent (triggered by CI):

1. Verifies deployment succeeds
2. Runs Playwright smoke tests against staging (3-5 paths)
3. Checks error monitoring for new error types
4. Posts summary to PR: "Deploy verified, staging smoke passed"

If smoke fails:

1. Debugger agent → root cause analysis
2. Posts diagnosis to PR
3. Alerts developer

Quality gate: Staging smoke must pass before feature is considered "shipped."

6. Day-to-Day Developer Experience

What a day looks like with the full agentic workflow:

Morning — check overnight CI:

- CI ran on 2 overnight PRs
- Automated PR review agent posted comments on both
- Developer reviews 2 structured summaries (not 400 lines of diffs)
- Approves or comments
- 15 minutes instead of 45

Mid-morning — new feature request:

Developer: "@claude, members need to see their upcoming roster assignments without logging into the admin app"

Orchestrator:

- PM agent: scopes the portal feature, writes user stories
- Staff Eng: proposes magic-link auth + read-only portal view
- SDET: identifies edge cases (expired links, no assignments, role leakage)
- Produces spec + plan in ~3 minutes

Developer reviews spec (5 minutes), approves.

Orchestrator delegates implementation:

- Worker A: portal API route (auto-audited by security hook)
- Worker B: portal page (auto-validated by intent hook)
- Worker C: magic link generation logic

Each worker self-validates as it goes. Hooks surface failures immediately. Developer watches progress in terminal, no intervention needed.

~25 minutes: orchestrator reports "ready for review, all hooks green"

Developer reviews PR summary (not code)

Merges.

Total human time: ~20 minutes for a feature that took 2 hours manually.

Afternoon — bug reported:

User: "Coordinator can see the Deactivate button on the People page"

Developer: "@claude there's a bug – coordinator sees deactivate button"

Debugger agent:

- Reads people/page.tsx canEdit logic
- Reads recent git diff
- Identifies: canEdit was changed from 3-condition to 2-condition check
- Root cause: null member was not guarded
- Proposes fix: restores null check

Developer approves fix direction.

Claude applies fix.

PostToolUse hook fires → intent test "Coordinator: does not see Deactivate" runs.

Test goes green immediately.

Pre-commit hook runs.

PR created: "fix: Coordinator canEdit null guard – closes #42"

Total time: 8 minutes.

7. Role Plays

7.1 Building a Feature

Feature: "Add a song category filter to the setlist builder"

Step 1 – ORCHESTRATOR RECEIVES TASK

Input: "Add song category filter to setlist builder so worship leaders can filter the song picker by category"

Step 2 – DEFINITION PHASE

Orchestrator invokes product-manager skill:

- User story: "As a Worship Leader, I want to filter songs by category so I can quickly find songs for a specific moment in the service"
- Acceptance criteria:
 - Filter appears above song list in setlist builder
 - Selecting a category shows only matching songs
 - Clearing filter restores full list
 - Filter state does not persist after modal close
- Non-goals: multi-category filter, saved presets

Step 3 – ARCHITECTURE PHASE

Orchestrator invokes staff-engineer skill:

- No API changes needed (songs already include categories)
- UI-only change: add category <select> to setlist page
- canAddSong permission gate must not change
- Files: src/app/admin/setlist/page.tsx only

Orchestrator invokes SDET skill:

- Test: filter shows correct songs for category
- Test: filter clears correctly
- Test: all roles see filter (it's read-display, not write)
- Test: empty category → "No songs match this filter" shown
- Risk: filter state leaking to next modal open

Step 4 – IMPLEMENTATION

Worker (UI):

- Reads current setlist/page.tsx
- Adds category <select> with useMemo filtered list
- No new API calls, no role gating needed
- PostToolUse hook fires → intent guard runs setlist journey tests
- All pass (no regression)

Step 5 – TESTING

Test Worker:

- Writes __tests__/intent/setlist.intent.test.tsx
- Adds 3 tests: filter shows correct songs, clear restores list, empty filter shows message
- Extends mapper.ts: setlist/page.tsx → setlist.intent.test.tsx
- All 3 new tests pass

Step 6 – PRE-COMMIT

- Husky runs npm run test:intent → 25 tests, all pass

Step 7 – PR

- "feat: add category filter to setlist song picker"
- Agent summary: 1 file changed, 3 new intent tests, no API surface, no role changes
- CI green

- Human approves in 2 minutes (no security concerns, clear scope)

Total human time: ~10 minutes of review across the whole feature.

Quality gates that fired:

- Layer 1: CLAUDE.md rules (canEdit pattern, no placeholder comments)
- Layer 2: PostToolUse intent guard (setlist journey tests after every edit)
- Layer 3: Pre-commit intent tests
- Layer 4: CI full suite

7.2 Fixing a Bug

Bug: "Roster page freezes (dark screen) when saving draft with a member who has no role assigned"

Step 1 – DEBUGGER AGENT ACTIVATED

Input: "Saving roster draft causes dark screen when a member has no role"

Debugger reads:

- src/app/admin/roster/page.tsx → handleSaveDraft function
- Finds: assignments.map(a => a.role.id) – no null guard on a.role
- Finds: no try/catch around the save handler
- Finds: setSaving(false) is BEFORE await loadRoster()

Root cause diagnosis:

"Three violations of CLAUDE.md Rule 7:

1. a.role.id throws TypeError when role is null (dark screen trigger)
2. No try/catch → uncaught throw activates React error overlay
3. setSaving(false) before await → button stays loading if reload throws"

Proposed fix:

1. Filter: assignments.filter(a => a.role !== null).map(a => a.role!.id)
2. Wrap entire handler in try/catch/finally
3. Move setSaving(false) to finally block

Step 2 – HUMAN APPROVES FIX DIRECTION

Developer: "Yes, apply it"

Step 3 – CLAUDE APPLIES FIX

- Edits handleSaveDraft in roster/page.tsx
- PostToolUse hook fires → intent guard runs roster journey tests:
 - PASS: Admin: roster page renders without crashing
 - PASS: Coordinator: edit controls visible
 - PASS: Musician: no edit controls
- No regression

Step 4 – ASYNC HANDLER AUDITOR

- (Auto-triggered by PostToolUse on roster/page.tsx handler)
- Confirms: try/catch/finally correct, setBusy in finally, no alert() usage

Step 5 – PRE-COMMIT

- 22 intent tests pass
- Commit: "fix: guard roster save against null role – prevents dark screen #57"

Step 6 – PR

- Agent summary: 1 function fixed, null guard added, try/catch/finally corrected
- Debugger diagnosis included in PR body as context

- CI green
- Human approves in 2 minutes

Total human time: 5 minutes. Bug that would have taken 30 minutes to diagnose manually.

Quality gates that fired:

- Layer 1: CLAUDE.md Rule 7 (async handler pattern) caught by debugger
- Layer 2: PostToolUse intent guard + async-handler-auditor
- Layer 3: Pre-commit
- Layer 4: CI

7.3 Testing a Feature

Scenario: New "Handbook" edit permission feature was added. SDET agent is invoked to ensure quality.

Step 1 – SDET AGENT RECEIVES SCOPE

Input: "Handbook page now has role-based edit permissions.
Coordinator, WorshipLeader, MusicCoordinator can edit.
Admin can toggle which roles have access in Settings."

Step 2 – SDET GENERATES TEST MATRIX

Unit tests needed:

- `hasHandbookEditPermission(role, handbookEditorRoles): boolean`
 - Admin always has permission
 - Role in list → true
 - Role not in list → false
 - Empty list → only Admin can edit

Integration tests needed:

- `PUT /api/settings/handbook-permissions`
 - Admin can update → 200
 - Non-Admin → 403
 - Tenant-scoped: update only affects requesting tenant

Intent tests needed:

- "Admin: can toggle Coordinator handbook edit permission in Settings"
- "Coordinator: sees edit controls on handbook when permission granted"
- "Coordinator: does not see edit controls when permission revoked"
- "WorshipLeader: sees edit controls when in permitted roles list"
- "Musician: never sees edit controls regardless of settings"
- "Settings: handbook permission section renders for Admin"

Edge cases flagged:

- What if `handbookEditorRoles` array is null (DB migration not run yet)?
- What if the same role appears twice in the array?
- What if an Admin removes Admin from the permitted list? (should be prevented)
- Multi-tenant: org A granting Coordinator access should not affect org B

Step 3 – SDET WRITES TESTS

Creates:

- `__tests__/unit/handbook-permissions.test.ts` (4 unit tests)
- `__tests__/integration/handbook-permissions-route.test.ts` (6 integration tests)
- Adds to `__tests__/intent/settings.intent.test.tsx` (3 new tests)
- Extends `mapper.ts: api/settings/handbook-permissions` → `settings.intent.test.tsx`

Step 4 – SDET RUNS AND REPORTS

All 13 new tests: PASS
Existing suite: 705 → 718 tests, all pass

Report to orchestrator:
"Feature tested. 13 tests added. Edge cases covered:
null permissions array guarded, duplicate role entries deduplicated.
One OPEN RISK: Admin-removes-Admin guard is UI-only (not server-side).
Recommend: add server-side validation in PUT /api/settings/handbook-permissions
to reject requests where Admin is removed from the permitted roles array."

Step 5 – ORCHESTRATOR ROUTES OPEN RISK

- Creates task: "Add server-side Admin-always-permitted guard in handbook permissions route"
- Assigns to implementation worker before PR is raised
- Worker implements, server-side guard added
- SDET adds one more integration test: attempts to remove Admin → 400 response
- All tests green

Quality gates that fired:

- Layer 1: SDET agent caught the missing server-side guard before it shipped
- Layer 2: PostToolUse intent guard on settings page
- Layer 2: PostToolUse security auditor on handbook-permissions API route
- Layer 3: Pre-commit (all intent tests)
- Layer 4: CI full suite

8. Roadmap — How to Get There

Phase A — Close Layer 2 (In-Session Quality Gates)

Estimated: 1-2 days each

Item	What It Does	Priority
A1. PostToolUse intent guard	Intent tests run after every src/ edit. Blocks Claude on failure.	Now
A2. PostToolUse API security hook	Tenant-security-auditor fires after every api/ edit	Next
A3. PostToolUse modal guard	Modal-regression-guard fires after any page.tsx edit	Next
A4. PostToolUse migration guard	Migration-safety-reviewer fires after any .sql edit	Next

Implementation: `.claude/settings.json` + `.claude/hooks/` shell scripts using `shadow-agent/mapper.ts` already built.

Phase B — Wire Existing Agents to Events

Estimated: 1 day

Item	What It Does
B1. Document orchestration pattern	Write <code>.claude/context/ORCHESTRATION-PATTERNS.md</code> as repeatable playbook
B2. Feature planning pipeline discipline	Make it a habit: every feature starts with pipeline, not ad-hoc coding
B3. Expand intent test coverage	Cover all 11 admin pages (currently 5)

B4. Add ESLint to pre-commit	Currently only in CI — should fail earlier
------------------------------	--

Phase C — Automated PR Workflow

Estimated: 2-3 days

Item	What It Does
C1. CI agent on PR	GitHub Action calls Claude API → posts structured review comment
C2. PR impact script	Maps PR diff → test suites → runs only affected tests in CI for speed
C3. Agent-generated PR body	Orchestrator writes PR description including agent reports

Phase D — Orchestrator Pattern

Estimated: 1 week

Item	What It Does
D1. Supervisor agent prompt	Write the orchestrator agent definition: how it plans, delegates, monitors
D2. Worker agent definitions	Scoped agents: API worker, UI worker, Test worker, DB worker
D3. Inter-agent protocol	How workers report status back to orchestrator (structured output)
D4. Human escalation rules	Define what requires human approval (migrations, destructive ops, scope changes)

Phase E — Staging Validation

Estimated: 2-3 days

Item	What It Does
E1. 5 Playwright smoke tests	Login, member create, roster view, coordinator blocked, logout
E2. Post-deploy agent	Runs smoke after each deploy, posts result
E3. Staging vs prod parity check	Agent verifies migration state matches expectations

Priority Order for THIS Project as Learning Bed

Week 1:	A1 (intent guard hook)	← highest quality impact, easiest to show team
Week 1:	A2 (security hook)	← demonstrates agentic quality control concept
Week 2:	B1 (document patterns)	← foundation for teaching the team
Week 2:	B3 (full intent coverage)	← 22 → 50+ intent tests
Week 3:	A3, A4 (modal + migration)	← complete Layer 2
Week 4:	C1 (CI PR review agent)	← visible to team in GitHub
Month 2:	D (orchestrator pattern)	← the ambitious target
Month 3:	E (staging validation)	← production confidence

9. Team Introduction Playbook

The 3 Things to Show First

- 1. The intent guard hook in action (Day 1)** Show the team: Claude edits a file, breaks a user journey, the hook fires, Claude sees the failure and fixes it — all in one session without human intervention. This is the "aha moment" for most developers.
- 2. The pre-commit gate (Day 1)** Show the team: introduce a bug, try to commit, Husky blocks it with a clear failure message. This is the concept they already understand (CI) applied closer to the source.
- 3. The specialist agent invocation (Day 2)** Show the team: ask Claude to add an API route, watch the security auditor fire automatically. Show them the structured report. Explain: "The agent knows what questions to ask because we wrote the agent description."

The Mental Model to Teach

"In a traditional workflow, quality is a phase — you write code, then you test, then you review. In an agentic workflow, quality is a property of every action — every file edit triggers validation, every agent output is checked by another agent, and humans review summaries not diffs."

What the Team Needs to Contribute

For the agentic workflow to work at team scale, every team member needs to be able to:

- 1. Write clear agent descriptions (what the agent is, what it checks, what it outputs)
- 2. Write intent tests for every feature they add (the contract the agent validates against)
- 3. Trust the hooks and not bypass them (`--no-verify` is not allowed unless you understand why a test is legitimately wrong)
- 4. Review PR summaries generated by agents (learn to read agent reports, not just diffs)

The Risk to Manage

The biggest risk with agentic workflows at team scale is miscalibrated trust. Developers will either:

- Trust agents too much (assume the agent caught everything, skip manual judgement)
- Trust agents too little (dismiss agent reports as noise, bypass hooks)

The mitigation: start with hooks that are **demonstrably useful** (the intent guard catches real bugs in the first week), build trust empirically, then expand the surface area.

Summary — The Gap in One View

TODAY	TARGET
Agent skills: 9 personas	Orchestrator: plans whole features
Agents: 5 specialists	Workers: scoped autonomous executors
Tests: 705 unit/integration	Tests: 705 + full intent coverage
Intent tests: 22 (5 pages)	Intent tests: 50+ (all pages)
Pre-commit: intent tests	Pre-commit: intent + lint + migration
In-session quality: NONE	In-session: 4 PostToolUse hooks
PR review: manual	PR review: automated agent comment
Staging: none	Staging: 5 Playwright smoke tests
Human reviews: every diff	Human reviews: agent summaries
Quality gap: Layer 2 is empty.	Quality gap: CLOSED at every layer.
Bugs caught: pre-commit or later.	Bugs caught: within the same edit.

The infrastructure is 40% built. The remaining 60% is not more complexity — it is connecting what already exists through hooks, documented patterns, and disciplined invocation.